Lecture 9: Application Service

Maharishi International University

Department of Computer Science

M.S. Thao Huy Vu

Maharishi International University - Fairfield, Iowa

All rights reserved. No part of this slide presentation may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying or recording, or by any information storage and retrieval system, without permission in writing from Maharishi International University (MIU).

Agenda

- **Introduction to AWS Application Services**
- **Simple Notification Service (SNS)**
- **Simple Queue Service (SQS)**

Event-driven Architecture

- Publisher/Subscriber: Push Model
- Producer/Consumer: Pull Model

Introduction to AWS Application Services

- Automate workflows, process, data
- Handle communication between different components of cloud applications.
- Provide scalable and managed solutions that reduce the need for internal infrastructure management.

Introduction to AWS Application Services

- Categories:
  - Compute: Serverless Computing with AWS Lambda
  - Messaging and Orchestration: SNS, SQS, and Step Functions help in managing communication and coordination between different software components
  - Workflow Service: Coordinate work across distributed application components.

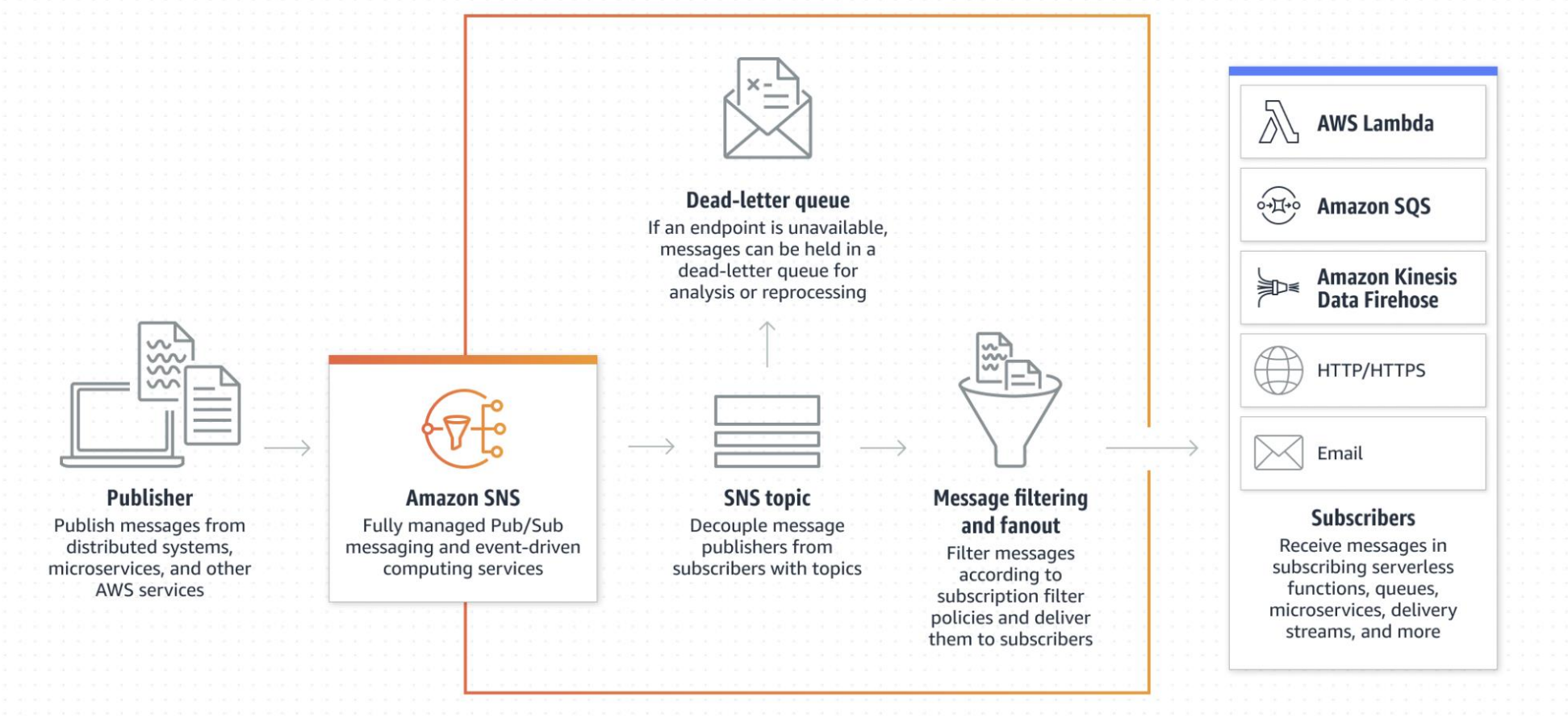
Introduction to AWS Application Services

- Benefits:
  - Scalability
  - Security
  - Cost-efficiency
  - Integration
  - Reliability

Amazon Simple Notification Service (SNS)

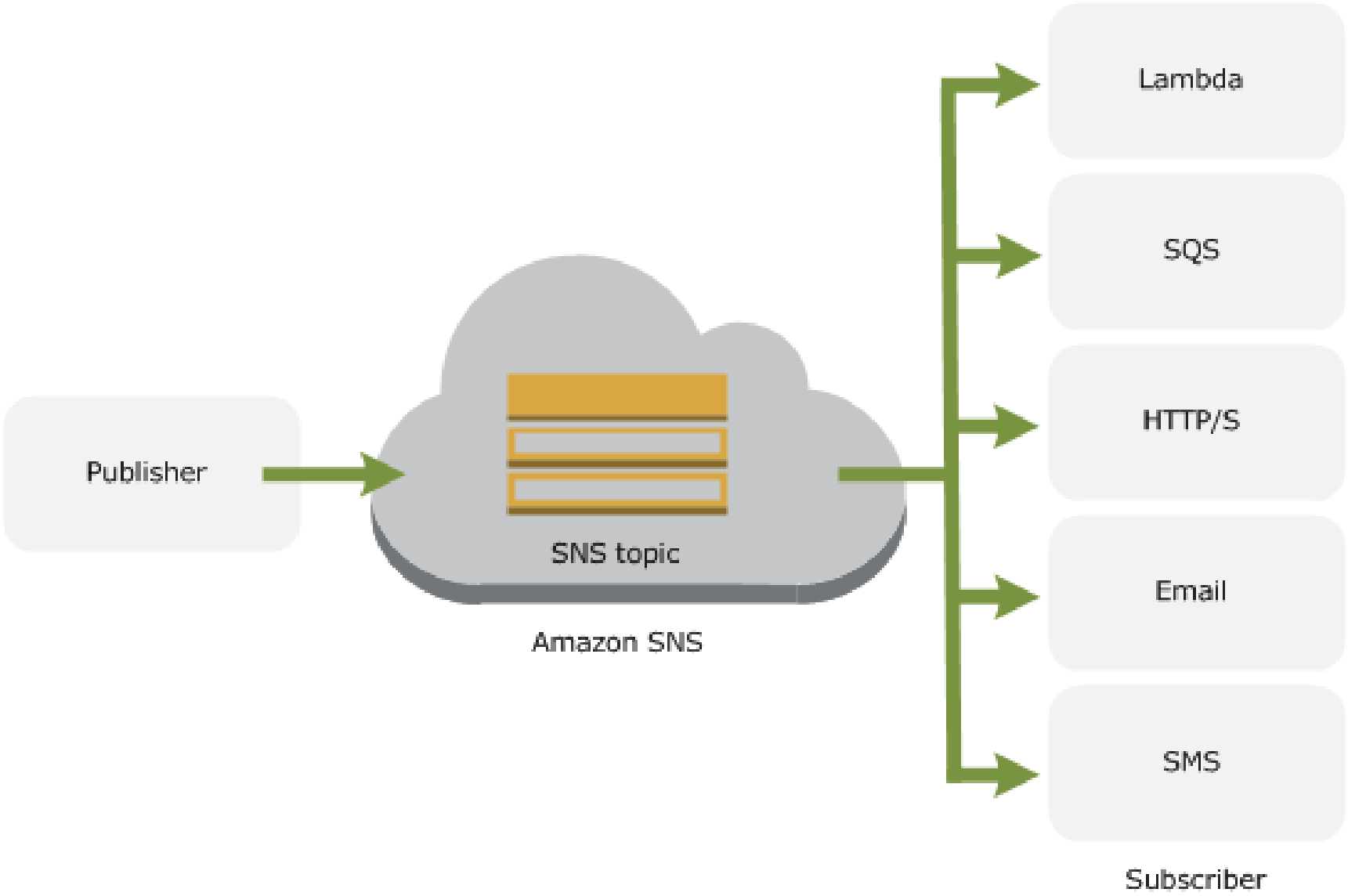
- SNS (Simple Notification Service): A fully managed service designed for high-throughput, automated communication between software components, microservices, and even-driven serverless applications.
- Able to fan out messages to a large number of subscribers in parallel.
- Support multiple transport protocols including SMS, email, SQS, and HTTP/S endpoints.

Amazon Simple Notification Service (SNS)



Amazon Simple Notification Service (SNS)

- Use cases:
  - Application and System Alerts
  - User Notifications
  - Workflow Systems
  - Decoupling Microservices


Lambda
SQS
HTTP/S
Email
SMS
Subscriber
Publisher
SNS topic
Amazon SNS

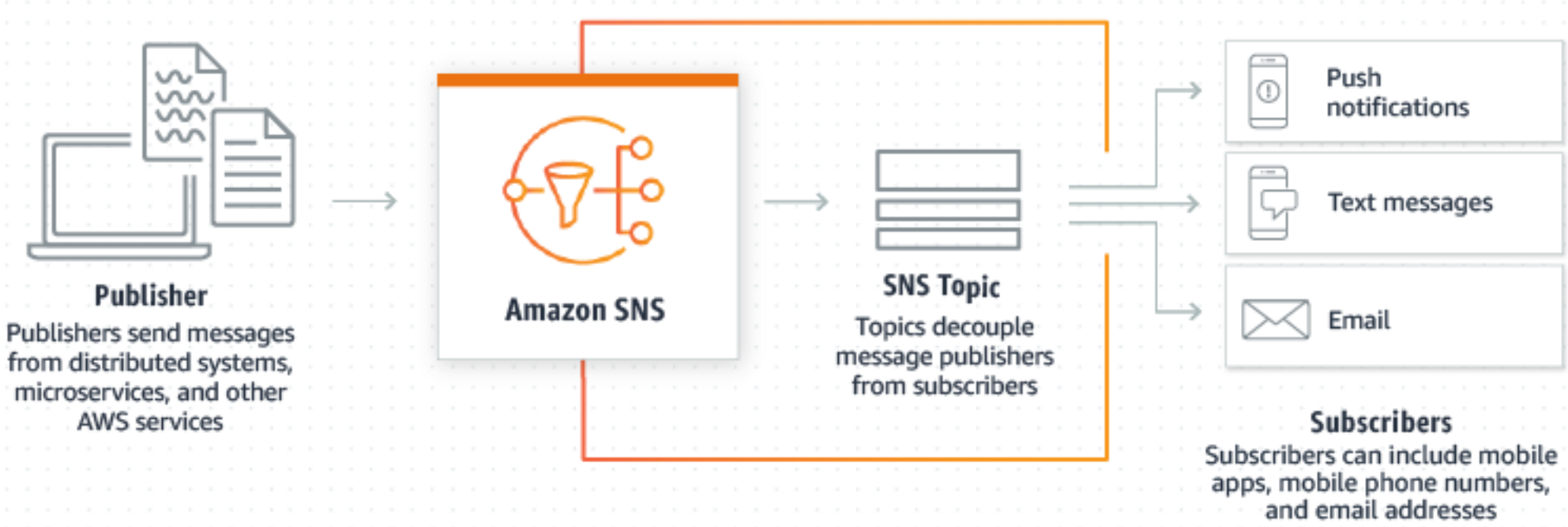
Application-to-application (A2A)

Amazon SNS lets you decouple publishers from subscribers. This is useful for application-to-application messaging for microservices, distributed systems, and serverless applications.



Application-to-person (A2P)

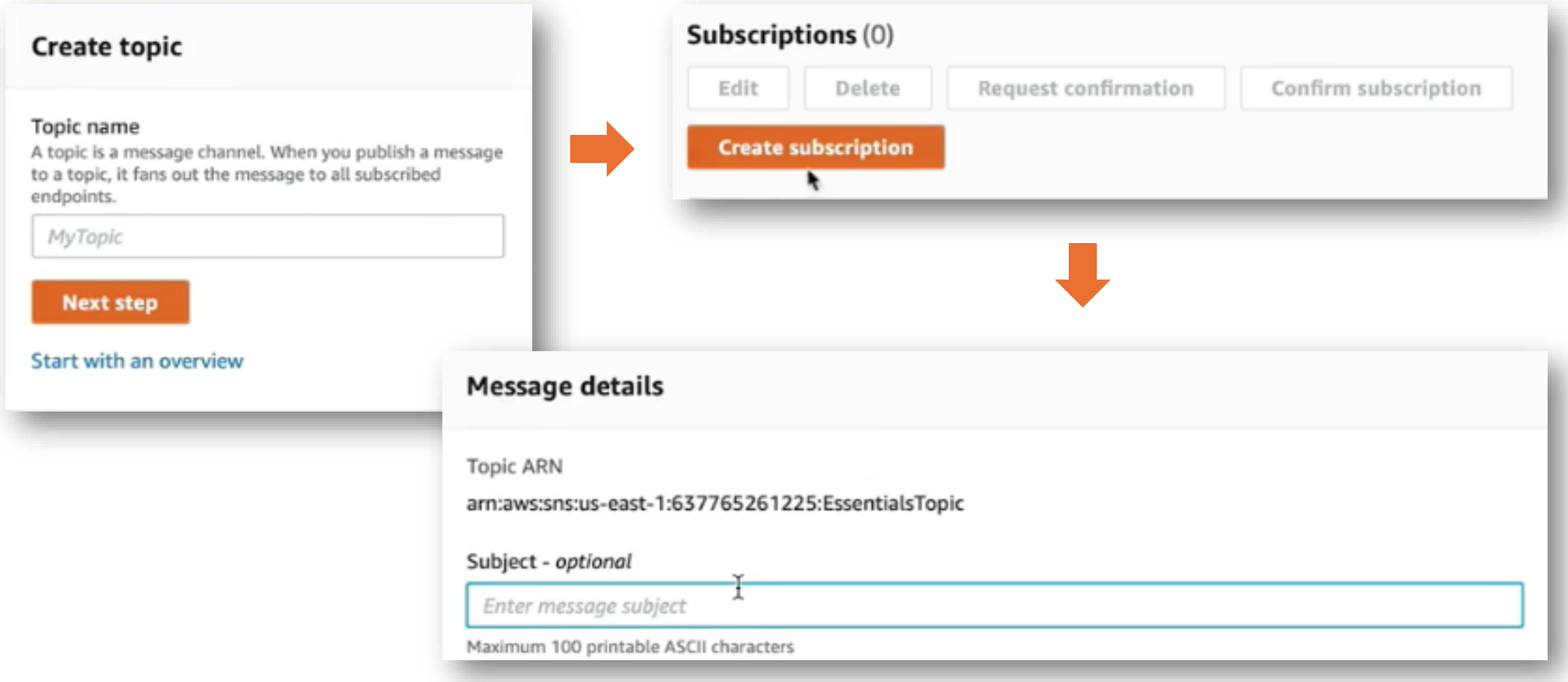
Amazon SNS lets you send push notifications to mobile apps, text messages to mobile phone numbers, and plain-text emails to email addresses. You can fan out messages with a topic, or publish to mobile endpoints directly.



SNS Basic Components

1. **Publishers** communicate **asynchronously** with subscribers by producing and sending a message to a topic.
2. **Topic** is a logical access point and communication channel.
3. **Subscribers** consume or receive the message or notification over one of the supported protocols when they are subscribed to the topic.

Create a Topic, Subscription, Publish



Message attributes

Message attributes let you provide any **arbitrary metadata** (such as timestamps, geospatial data, signatures, and identifiers) about the message.

You can use message attributes to make your messages filterable using subscription filter policies. You can apply **filter** policies to topic subscriptions. When a filter policy is applied, a subscription receives only those messages that have attributes that the policy accepts.

Sample SNS payload


```
{
   "Type": "Notification",
   "MessageId": "a1b2c34d-567e-8f90-g1h2-i345j67klmn8",
   "TopicArn": "arn:aws:sns:us-east-2:123456789012:MyTopic",
   "Message": "message-body-with-transaction-details",
   "Timestamp": "2019-11-03T23:28:01.631Z",
   "SignatureVersion": "4",
   "Signature": "signature",
   "UnsubscribeURL": "unsubscribe-url",
   "MessageAttributes": {
      "customer_interests": {
         "Type": "String.Array",
         "Value": "[\"soccer\", \"rugby\", \"hockey\"]"
      },
      "store": {
         "Type": "String",
         "Value":"example_corp"
      },
      "event": {
         "Type": "String",
         "Value": "order_placed"
      },
      "price_usd": {
         "Type": "Number",
         "Value":210.75
      }
   }
}
```

Amazon SNS message filtering

By default, an Amazon SNS topic subscriber receives every message published to the topic. To receive a subset of the messages, a subscriber must assign a filter policy to the topic subscription.

You need to pass the **message attributes** (MessageAttributes) parameter which is a map consists of a key and value.

A policy that accepts messages

If any single attribute in this policy doesn't match an attribute assigned to the message, the policy rejects the message.

```
"MessageAttributes": {
   "customer_interests": {
      "Type": "String.Array",
      "Value": "[\"soccer\", \"rugby\", \"hockey\"]"
   },
   "store": {
      "Type": "String",
      "Value":"example_corp"
   },
   "event": {
      "Type": "String",
      "Value": "order_placed"
   },
   "price_usd": {
      "Type": "Number",
      "Value":210.75
   }
}
```

Payload

```
{
   "store": ["example_corp"],
   "event": [{"anything-but": "order_cancelled"}],
   "customer_interests": [
      "rugby",
      "football",
      "baseball"
   ],
   "price_usd": [{"numeric": [">=", 100]}]
}
```

Policy

A policy that rejects messages

If any mismatches occur, the policy rejects the message. The encrypted attribute isn't present in the message, this causes the message to be rejected regardless of the value assigned to it.

```
"MessageAttributes": {
   "customer_interests": {
      "Type": "String.Array",
      "Value": "[\"soccer\", \"rugby\", \"hockey\"]"
   },
   "store": {
      "Type": "String",
      "Value":"example_corp"
   },
   "event": {
      "Type": "String",
      "Value": "order_placed"
   },
   "price_usd": {
      "Type": "Number",
      "Value":210.75
   }
}
```

Payload

```
{
   "store": ["example_corp"],
   "event": ["order_cancelled"],
   "encrypted": [false],
   "customer_interests": [
      "basketball",
      "baseball"
   ]
}
```

Policy

Message delivery status logging

Amazon SNS supports logging the delivery status of notification messages using CloudWatch. Amazon SNS sends the following metrics to CloudWatch:

- NumberOfMessagesPublished
- NumberOfNotificationsDelivered
- NumberOfNotificationsFailed
- NumberOfNotificationsFilteredOut
- PublishSize – Message size etc.

Based on these metrics, you can set a policy for ASG.

Unsuccessful message delivery

SNS retries if message is not delivered. Policies can be:

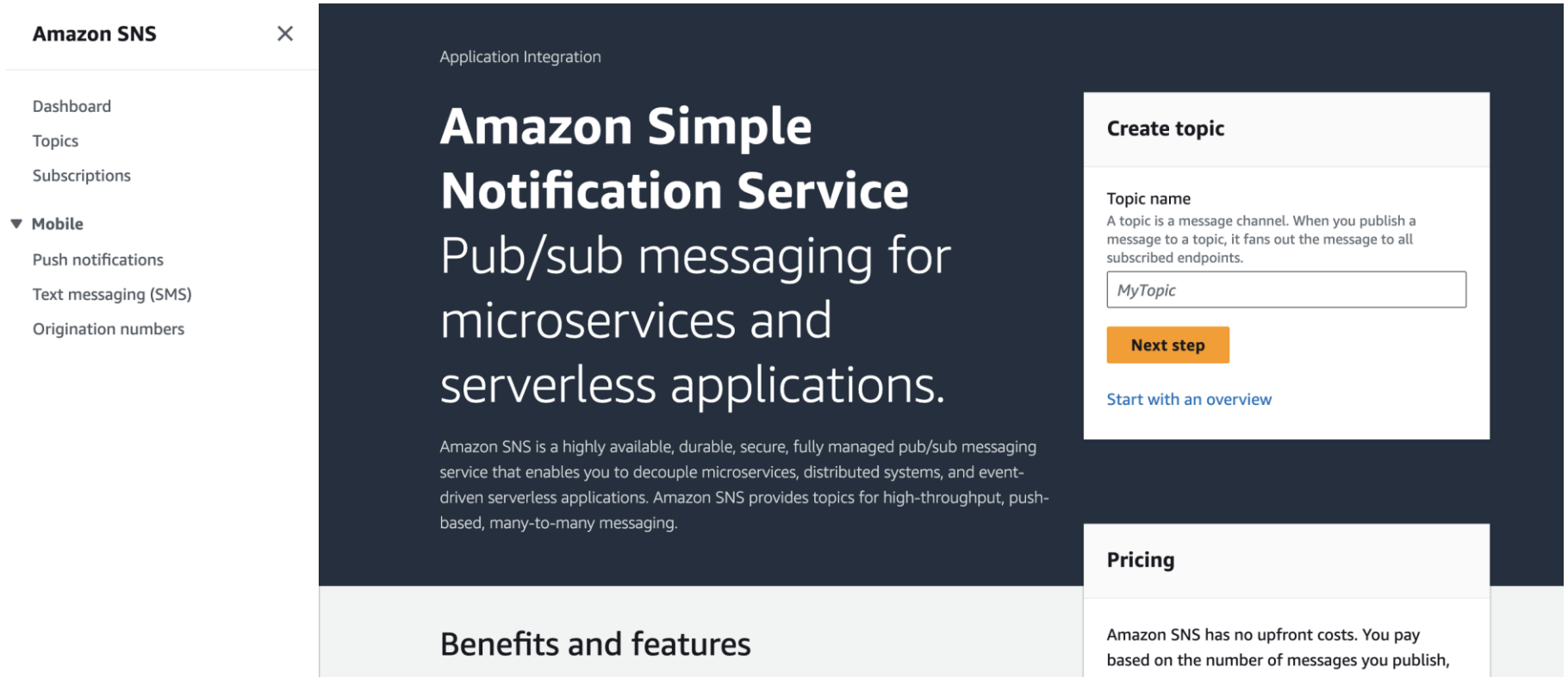
- 3 times immediately in the no-delay phase
- 2 times (1 second apart) in the pre-backoff phase
- 10 times (with exponential backoff from 1 second to 60 seconds)
- 35 times (60 seconds apart) in the post-backoff phase

If retry is not successful, the message is sent to the **dead-letter queue** further analysis or manual reprocessing.

You can have a dead-letter queue for SQS as well.

Amazon Simple Notification Service (SNS)

- Demo



Amazon Simple Queue Service (SQS)



Producer

Sends messages to Amazon SQS

Amazon SQS

Fully managed message queuing service to reliably and continually exchange any volume of messages from anywhere



Encryption

Messages are encrypted at rest (AWS KMS) and in flight with HTTPS/TLS



Consumers

Nearly infinite scalability and ability to increase message throughput to consumers without the need to preprovision capacity

Amazon Simple Queue Service (SQS)

- SQS is a message queuing service that decouples and scales microservices, distributed systems, and serverless applications.
- Reliability
- Scalability
- Able to send, store, and receive messages between software components at any volume without losing messages.

Amazon Simple Queue Service (SQS)

- Core Concepts:
  - Queue Types:
    - Standard Queues
    - FIFO Queues
  - Message Lifecycle
  - Visibility Timeout

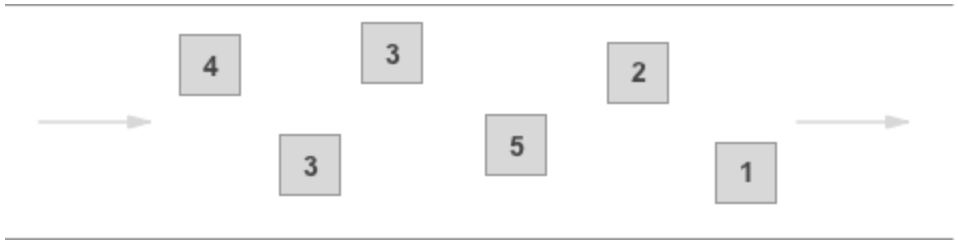
Amazon Simple Queue Service (SQS)

- Use cases:
  - Decoupling components
  - Load Balancing
  - Handling Spikes in Traffic

Standard queue

Use it when the throughput is important. Standard queues provide:

- **Unlimited throughput** - Standard queues support a nearly unlimited number of API calls per second. API calls include SendMessage, ReceiveMessage, DeleteMessage.
- **At-least-once delivery** - A message is delivered at least once, but occasionally more than one copy of a message is delivered.
- **Best-effort ordering** - Occasionally, messages are delivered in an order different from which they were sent.



FIFO queue

Use it when the order of events is important. FIFO queues guarantee consistency and strict message ordering.

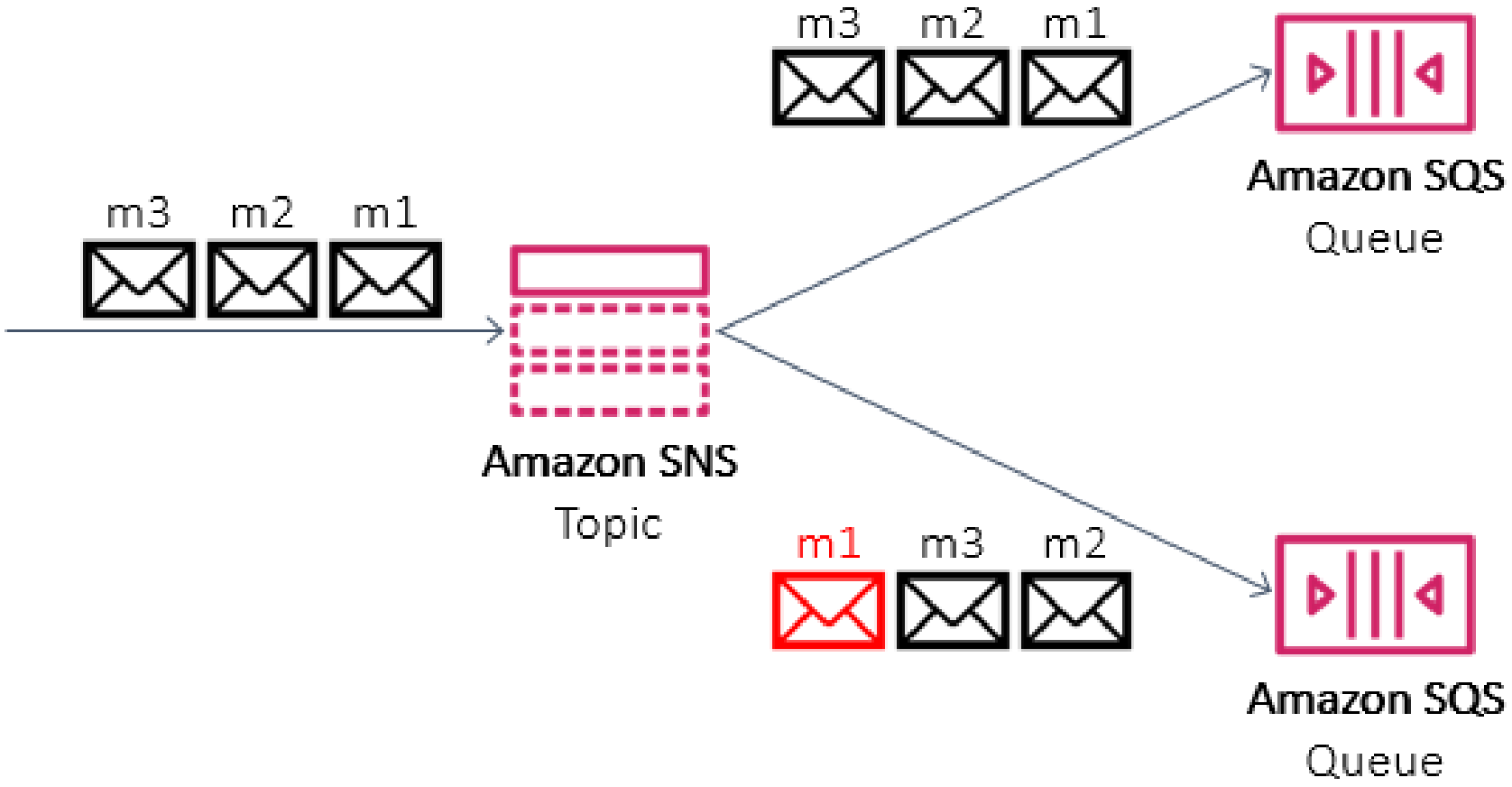
- **High** (Not Unlimited) **Throughput** - If you use **batching**, FIFO queues support up to 3,000 messages per second (300 API calls, 10 messages per call).
- **Exactly-Once processing** - A message is delivered once and remains available until a consumer processes and deletes it. Duplicates aren't introduced into the queue.
- **First-In-First-Out delivery** - The order in which messages are sent and received is strictly preserved.

Message ordering and deduplication concepts also apply to SNS.



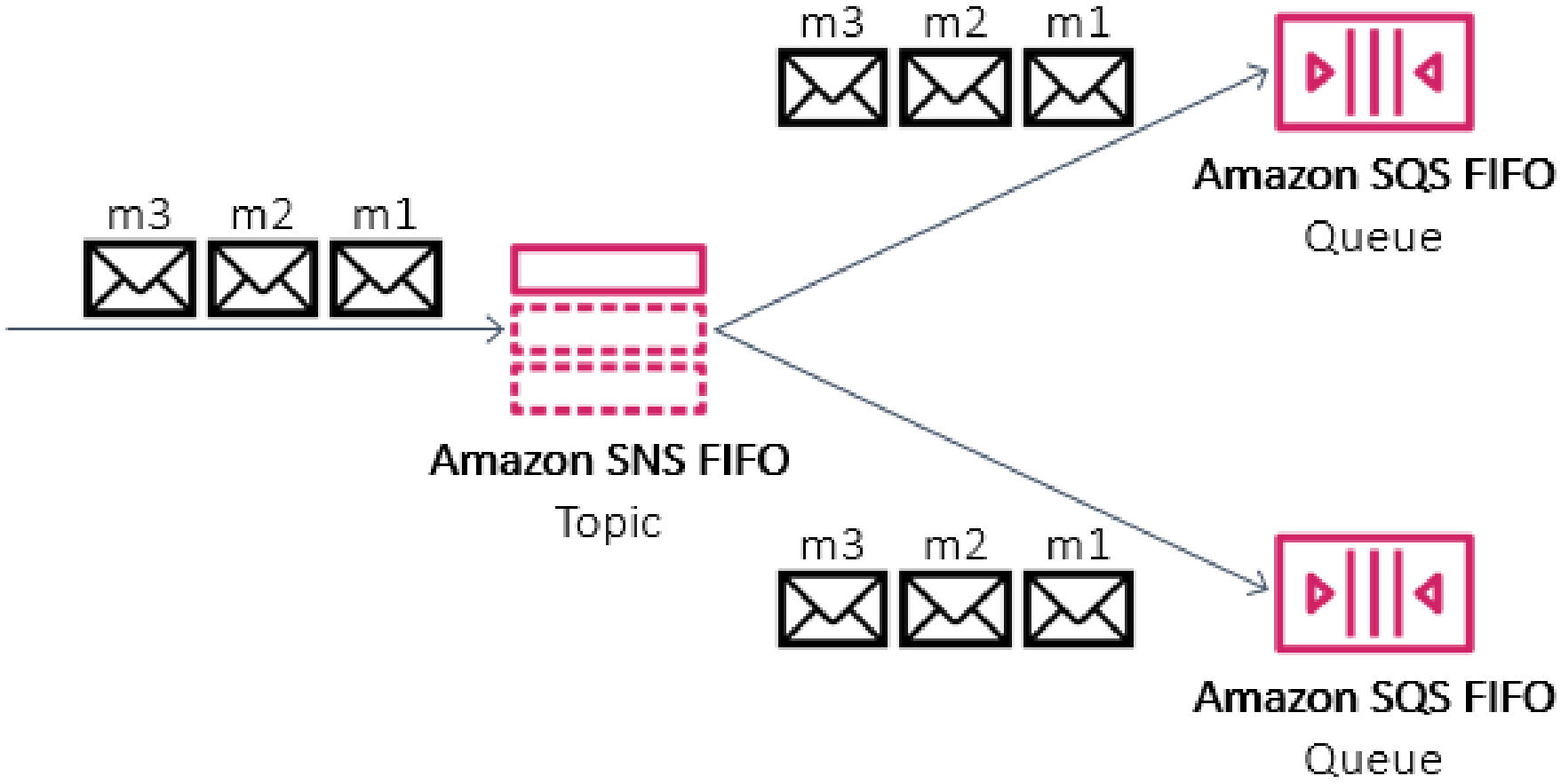
Best-effort message ordering

Standard SNS and SQS provide best-effort message ordering with rarely out of order delivery.



Strict message ordering

SNS FIFO and SQS FIFO provide strict message ordering.



Message deduplication ID

You ordered an item online. But you were charged twice. Because the app processed the message twice. That is when deduplication id (or idempotency key) comes into the picture.

The message deduplication ID is the token used for deduplication of sent messages. If a message with a particular message deduplication ID is sent successfully, any messages sent with the same message deduplication ID are accepted successfully but aren't delivered during the 5-minute **deduplication interval**.

If content-based deduplication for the queue is enabled. The producer can omit the message deduplication ID.

Deduplication id can also be used for:

- Different contents but must be treated as duplicates.
- Identical content but different attributes must be treated as unique.

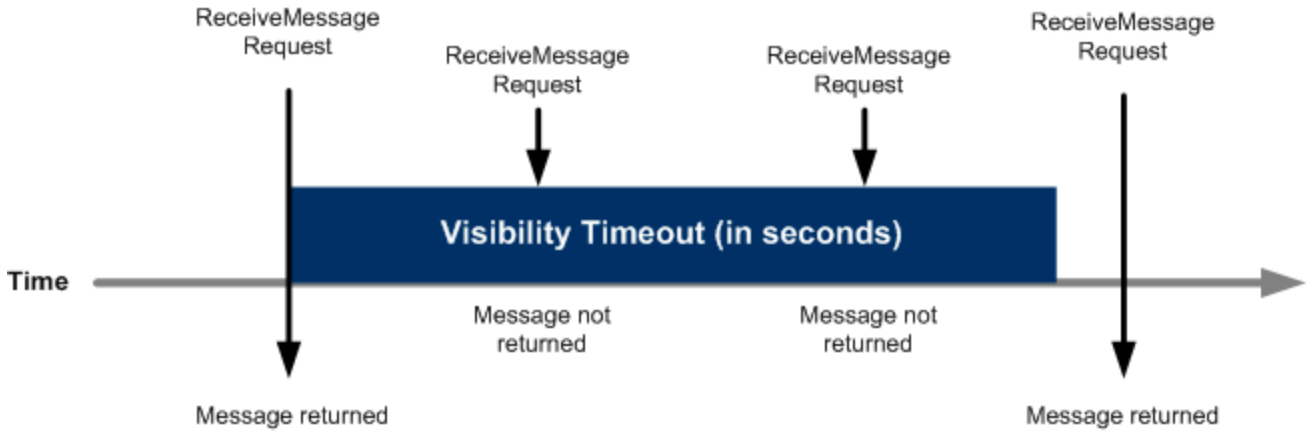
Message group ID

The message group ID is the tag that specifies that a message belongs to a specific message group. Messages that belong to the same message group are always processed one by one.

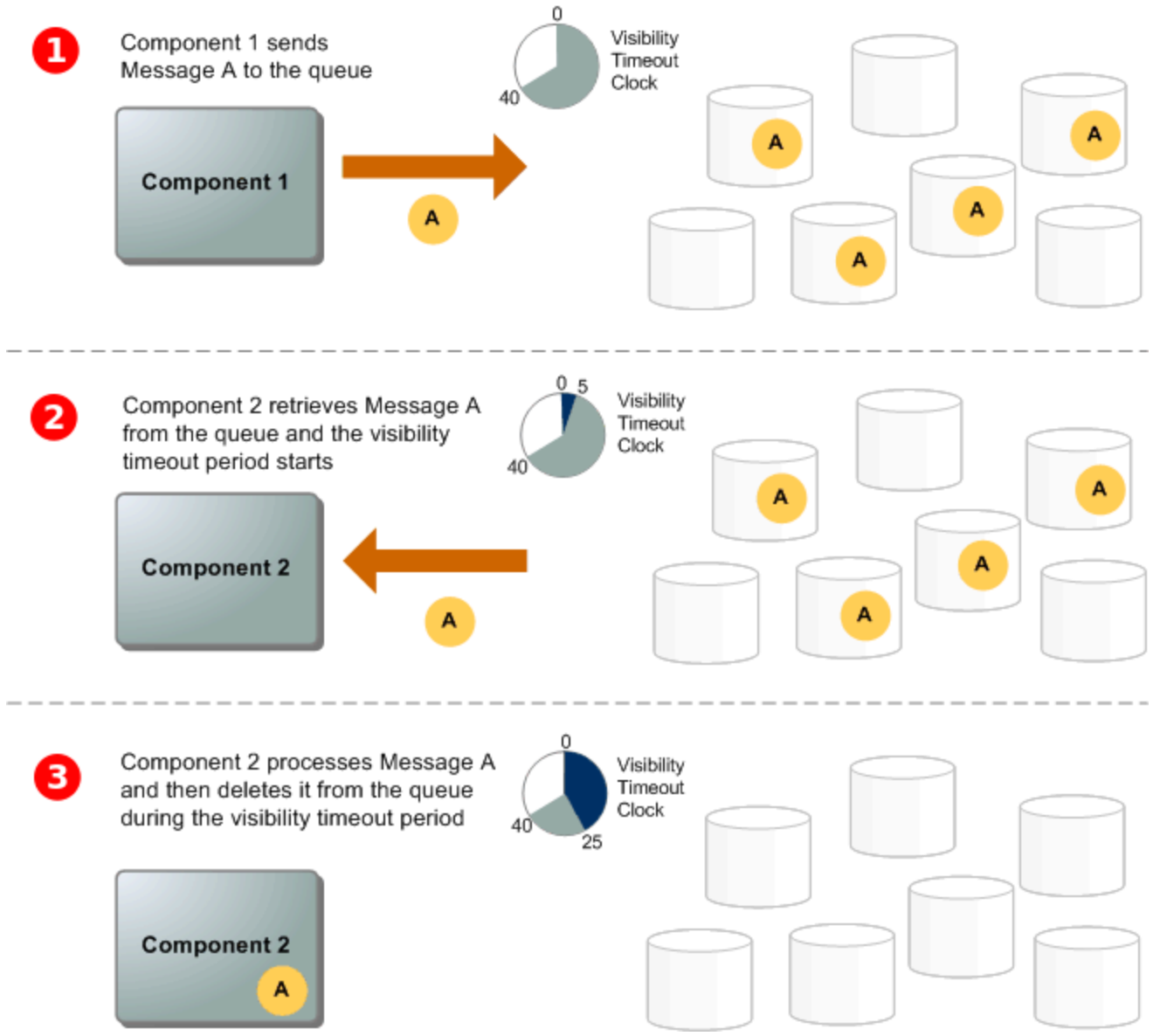
- Max number of messages in the queue is 20,000. Use group ID to avoid it. With groups, you can process messages parallelly.
- There could be bottleneck on the group of messages if the current message is not being processed properly by a consumer. Use group ID to prevent it.

SQS visibility timeout

When a consumer receives and processes a message from a queue, the message **remains** in the queue. SQS **doesn't automatically delete** the message. Because SQS is a **distributed system**, there's no guarantee that the consumer actually receives the message due to a connectivity issue, or an issue in the consumer application. Thus, the consumer **must delete** the message from the queue after receiving and processing it. Exceptions that Lambda and some AWS services automatically deletes messages.

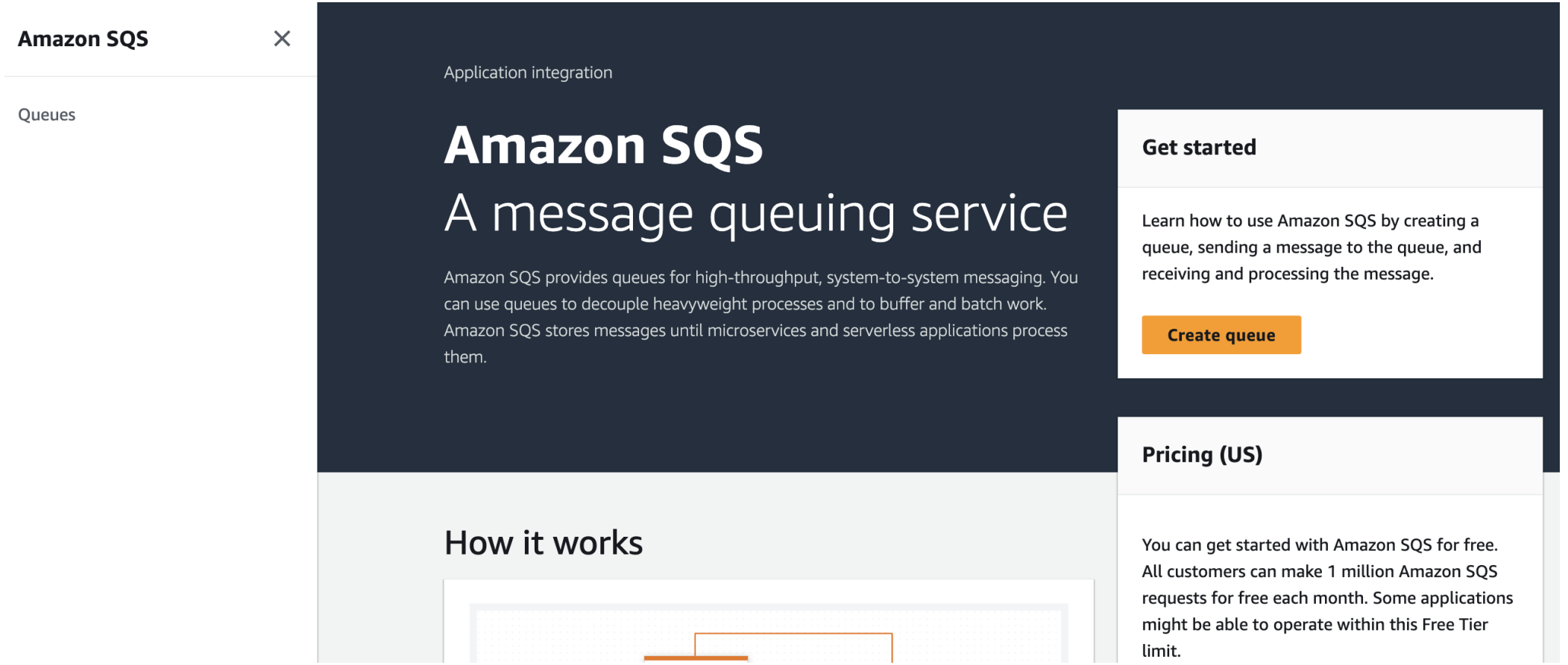


Message lifecycle

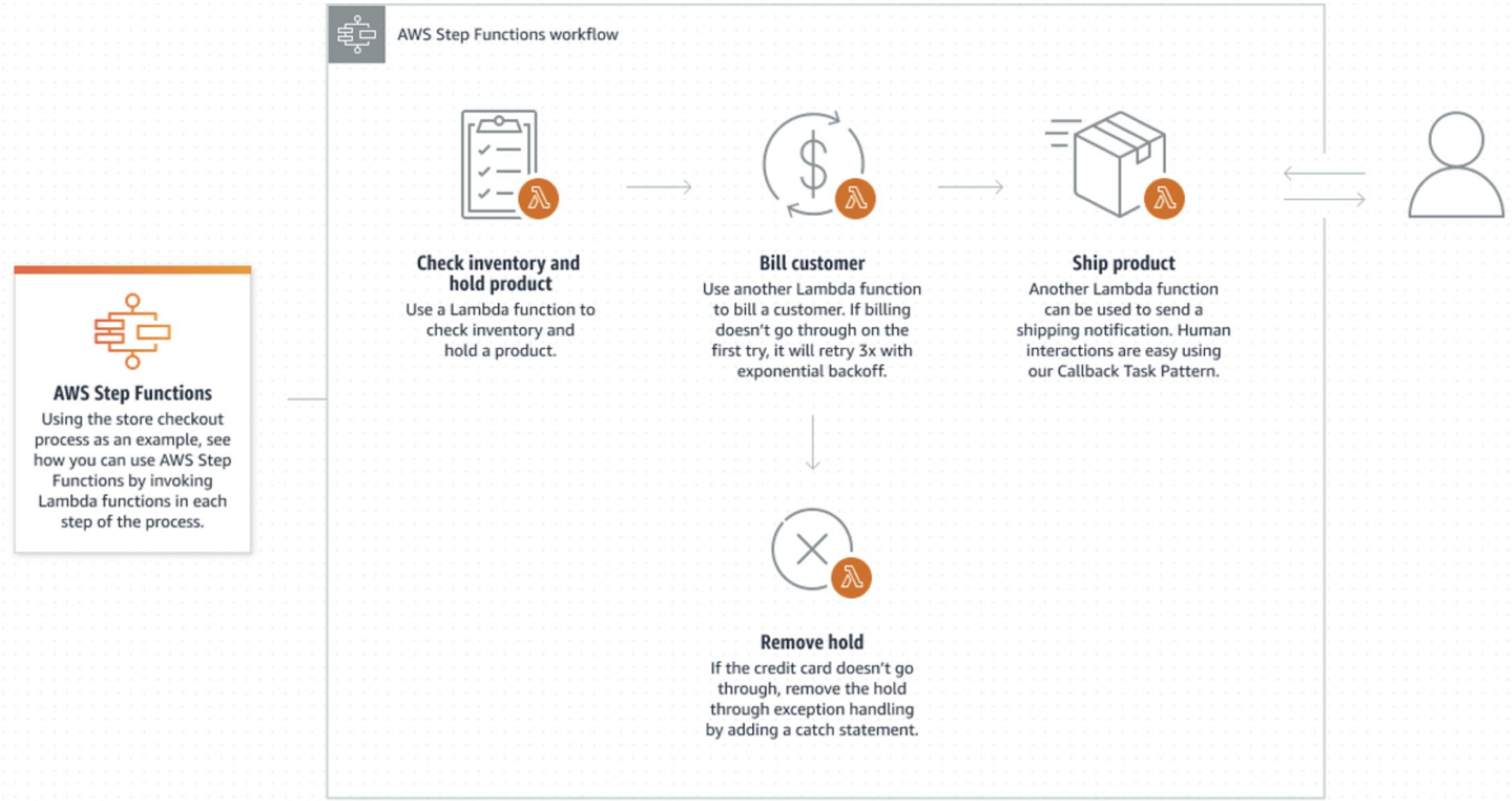


Amazon Simple Queue Service (SQS)

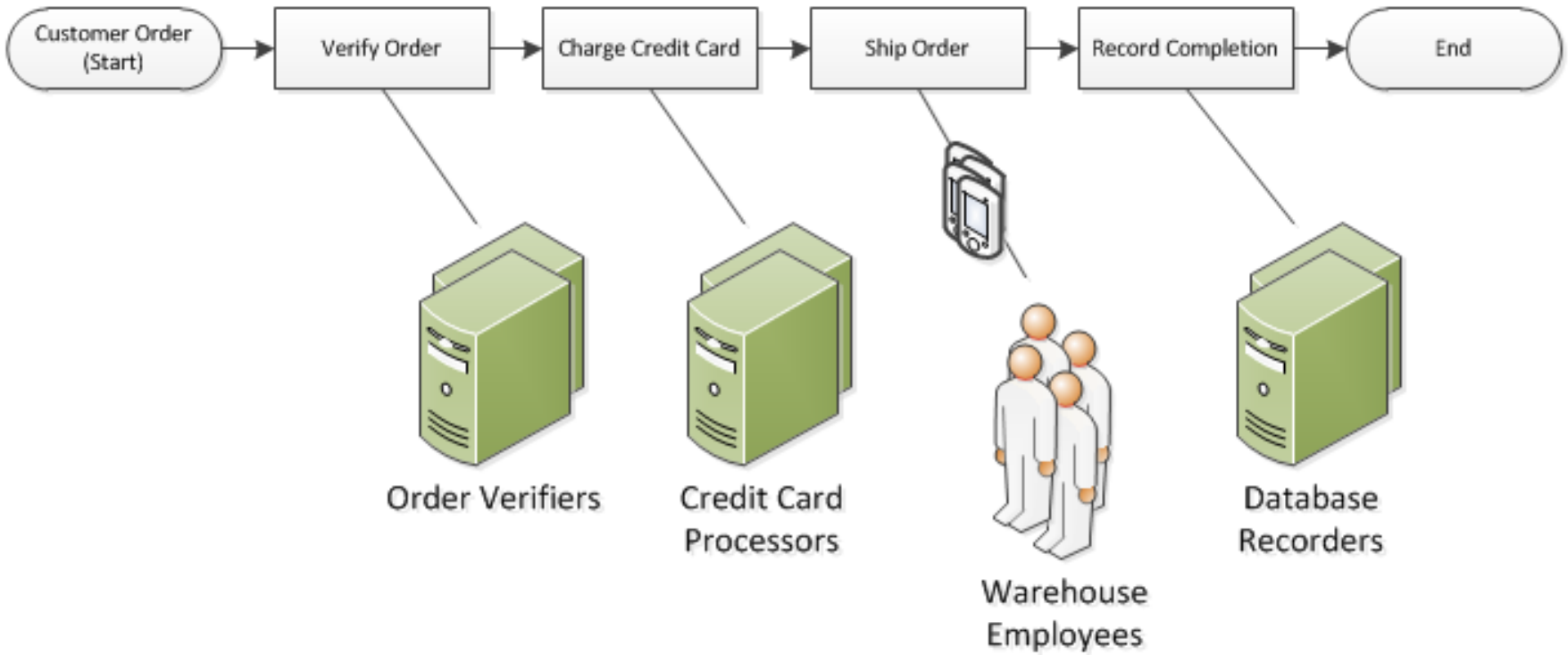
- Demo



AWS Step Functions



AWS Simple Workflow Service (SWF)



References

- https://docs.aws.amazon.com/
- ChatGPT: https://chatgpt.com/
- Google AI: https://gemini.google.com/app